

**A MAJOR PROJECT REPORT
ON**

“KeyDash”

**Submitted for the partial fulfillment of the Degree of Bachelor of Engineering in
Computer awarded by Pokhara University**



SUBMITTED BY:

Britant Shrestha (22070449)

Manish Shrestha (22070452)

Ranjana Dhakal (22070457)

Rijan Shrestha (22070458)

SUPERVISOR:

Er. Shiva Kunwar

Affiliated to Pokhara University

**DEPARTMENT OF COMPUTER ENGINEERING
POKHARA ENGINEERING COLLEGE**

Phirke, Pokhara-8

2026

CERTIFICATE

Date: 2026/07/24

This is to certify that the project entitled “**KeyDash**” has been carried out by Britant Shrestha (22070449), Manish Shrestha (22070452), Ranjana Dhakal (22070457) and Rijan Shrestha (22070458) in partial fulfillment of the degree of Bachelor of Computer Engineering of Pokhara University, during the academic year 2026. To the best of our knowledge and belief, this work has not been submitted elsewhere for the award of any other degree and thus has been accepted.

Er.
External Examiner

Er. Shiva Kunwar
Supervisor

Er. Suraj Basant Tulachan
HOD, Computer & Electronics

Er. Krishna Ghimire
Co-Ordinator, Research Management Cell

ACKNOWLEDGEMENT

We would like to extend our heartfelt gratitude to everyone who supported and guided us throughout the completion of our project. First and foremost, we are profoundly thankful to our respected supervisor, **Er. Shiva Kunwar**, for his expert guidance, continuous encouragement, and constructive suggestions at every stage of this project. His patience and dedication to our academic growth significantly shaped both the technical and practical direction of our work.

We also express our sincere appreciation to the **Department of Computer Engineering** for providing a strong academic foundation and the essential resources that made this project achievable. The department's curriculum, lab facilities, and regular project evaluations were instrumental in our learning process.

We would like to acknowledge the contributions of all faculty members and technical staff, whose teachings and timely assistance helped us overcome many challenges during the development phase. Their expertise and support added great value to our learning experience.

Special thanks to our classmates and friends who helped with early testing, shared ideas, and gave us constructive feedback that improved the overall quality of our project.

Lastly, we want to recognize the collective effort of our group. This project would not have been possible without our collaboration, mutual understanding, and commitment toward achieving a common goal.

Britant Shrestha (22070449)

Manish Shrestha (22070452)

Ranjana Dhakal (22070457)

Rijan Shrestha (22070458)

Table of Contents

CERTIFICATE.....	i
ACKNOWLEDGEMENT.....	ii
Chapter 1 Introduction.....	2
1.1 Background.....	2
1.2 Theory.....	2
1.2.1 React Framework.....	2
1.2.2 Supabase Backend.....	2
1.2.3 PostgreSQL.....	3
1.2.4 User Authentication.....	3
1.2.5 Cloud Storage.....	3
1.2.6 Data Modeling.....	3
1.2.7 State Management in React.....	3
1.2.8 Version Control with GitHub.....	4
1.2.9 Security Considerations.....	4
1.3 Project Category (web application).....	4
1.4 Detailed Problem Definition.....	5
1.5 Presently Available System for the Same Purpose.....	5
1.6 Objective of the Project.....	6
1.7 Scope.....	6
Chapter 2: Literature Review.....	7
2.1 Introduction.....	7
2.2 Content Review.....	7
2.3 Traditional Review.....	7
2.4 Our Proposed System.....	7
Chapter 3: Analysis.....	8
3.1 Project Management.....	8

3.1.1 Initial Investigation.....	8
3.1.2 Feasibility Analysis.....	8
3.1.3 Cost Benefit Analysis.....	9
3.2 Requirement Analysis.....	9
Chapter 4: Design.....	11
4.1 Software Requirements Specification.....	11
4.1.1 Functional Requirements.....	11
4.1.2 System Specification.....	11
4.1.3 Non-functional Requirements.....	12
4.2 Risk Assessment.....	12
Chapter 5: System Modeling.....	14
5.1 Block Diagram.....	14
5.2 UML Diagram.....	15
5.2.1 UML Use Case Diagram.....	15
5.3 Flowcharts.....	16
5.3.1 System Flowchart.....	16
5.4 System Workflow.....	17
5.4.1 Registering Users.....	18
5.4.2 Log In Option.....	18
5.4.3 View modes and respective levels.....	18
5.4.4 View Leaderboard.....	18
5.4.5 View Scoreboard.....	18
5.4.6 Profile Section.....	19
5.5 Data Flow Diagram.....	20
5.5.1 Context Level DFD.....	20
5.5.2 Level-1 DFD.....	20
5.6 Database Design.....	21
5.6.1 ER (Entity Relationship) Diagram.....	21
5.6.2 Supabase Database Table for users.....	22
5.6.3 Supabase As a Database.....	24

Chapter 6: Coding.....	26
6.1 Hardware Specification.....	26
6.2 Platform.....	26
6.3 Technology Used.....	26
6.3.1 React Technology.....	26
6.4 Programming Language Used.....	27
6.4.1 JavaScript Programming Language.....	27
6.4.2 SQL (Structured Query Language).....	28
6.5 Software Tools Used.....	28
6.5.1 Visual Studio Code.....	28
6.5.2 Git and GitHub.....	28
6.5.3 Web browser.....	29
6.5.4 Adobe Illustrator.....	29
6.6 Coding Style Followed.....	29
Chapter 7: Testing.....	31
7.1 Test Plan.....	31
7.2 Test Cases.....	31
7.2.1 Testing Cases Summary.....	31
7.2.2 Sample Test Cases.....	32
Chapter 8: Limitations And Recommendations.....	38
8.1 Limitations.....	38
8.2 Recommendation.....	38
Chapter 9: Conclusion.....	39
9.1 Conclusion.....	39
REFERENCES.....	40

FIGURE INDEX

S.N.	FIGURE	Page No.
1.	Figure 5.1: Block Diagram of the System.....	14
2.	Figure 5.2: UML Use Case Diagram.....	15
3.	Figure 5.3: System Flowchart.....	16
4.	Figure 5.4: System Workflow.....	17
5.	Figure 5.5: Context Level DFD.....	20
6.	Figure 5.6: Level-1 DFD.....	20
7.	Figure 5.7: ER Diagram.....	21
8.	Figure 7.1: Test case 1.....	31
9.	Figure 7.2: Test case 2.....	32
10.	Figure 7.3: Test case 3.....	33
11.	Figure 7.4: Test case 4.....	34
12.	Figure 7.5: Test case 5.....	35
13.	Figure 7.6: Test case 6.....	36
14.	Figure 7.7: Test case 7.....	37

TABLE INDEX

S.N.	TABLE	Page no.
1.	Table 5.1: Supabase Table Profile.....	22
2.	Table 5.2: Supabase Table Leaderboard_sentence.....	23
3.	Table 5.3: Supabase Table Leaderboard_timed.....	23
4.	Table 7.1: Testing Cases Summary.....	29
5.	Table 7.2: Test Case 1.....	31
6.	Table 7.3: Test Case 2.....	31
7.	Table 7.4: Test Case 3.....	31
8.	Table 7.5: Test Case 4.....	32
9.	Table 7.6: Test Case 5.....	33
10.	Table 7.7: Test Case 6.....	34
11.	Table 7.8: Test Case 7.....	35

APPENDICES

Appendix A: Login screen.....	39
Appendix B: Sentence mode.....	39
Appendix C: Timed mode.....	40
Appendix D: Leaderboard.....	40
Appendix E: Profile Section.....	41
Appendix F: Profile Update Setting.....	42
Appendix G: Backend Database.....	43
Appendix H: Daily challenge.....	44
Appendix I: AI Battle.....	45

ABSTRACT

“**KeyDash**” is an interactive and user-friendly web application designed to measure typing speed and accuracy in real-time. It is developed as a major project for the partial fulfillment of degree of Bachelor of Engineering in Computer Engineering of Pokhara University. The project has been developed by us with a great support of our college administration, faculties and friends and is liable to be submitted to the Department of Computer Engineering College, Phirke – 8, Pokhara, Nepal.

“**KeyDash**” provides an engaging platform for users to test and improve their typing skills across multiple modes across language mode (Nepali/English), Coding mode, Symbols Mode, and a custom mode for user-supplied text. The application features real-time performance metrics (Words Per Minute - WPM, accuracy, and error count), daily streaks system to build consistent practice habits, and over 1200+ tiered achievements across speed, streaks, coding, symbols, multiplayer and AI battle categories. Competitive features include daily, weekly, monthly, and all time leaderboards, real-time multiplayer typing races powered by Supabase Realtime and AI opponents at Easy, medium, Hard and Pro Difficulty. Users can also generate and share result cards on social media. It is built using modern web technologies and it ensures a seamless and responsive experiences across devices. The project emphasizes clean code practices, efficient algorithms for performance calculation, and an intuitive UI/UX design. It serves as both an educational tool, demonstrating proficiency in full-stack web development.

The software development team is really appreciative to our teacher, supervisor, and friends for their invaluable assistance and cooperation in bringing this project to life. The process of creating this project has been fantastic, and we hope that our users will feel the same way.

Chapter 1 Introduction

1.1 Background

In the current digital era, typing accuracy and speed are critical abilities for professionals, students, and regular computer users. Typing efficiently improves workflow overall, increases productivity and cuts down on time spent on documentation. By giving users immediate feedback in performance indicators including words per minute (WPM), accuracy and error rates, typing speed testers frequently used to evaluate and enhance these abilities.

Current typing test programs frequently don't include progress-tracking capabilities, real-time analytics, or a contemporary, user-friendly design. Many are too complicated or too simple, which deters users from using them. A well-designed, feature-rich, and responsive typing speed tester is required in order to provide a smooth user experience and assist users in monitoring their progress over time.

1.2 Theory

1.2.1 React Framework

React is an open-source JavaScript library developed by Meta (formerly Facebook) for building dynamic and interactive user interfaces. It follows a component-based architecture, enabling developers to create reusable UI elements that update efficiently via a virtual DOM. For KeyDash, React was chosen due to its performance optimizations, extensive ecosystem (e.g., hooks), and compatibility with modern web development practices. Its declarative nature simplifies rendering real-time typing metrics (e.g., WPM, accuracy) during tests.

1.2.2 Supabase Backend

Supabase is an open-source Backend-as-a-Service (BaaS) platform that provides PostgreSQL database management, authentication, and storage APIs. KeyDash leverages Supabase to handle user data, test results, and authentication, eliminating the need for custom server-side code. Supabase's RESTful and real-time APIs ensure

seamless integration with the React frontend, enabling features like historical performance tracking and multi-device synchronization.

1.2.3 PostgreSQL

PostgreSQL is a robust, open-source relational database system powering Supabase. It stores structured data for KeyDash, including user profiles, test histories, and typing analytics. PostgreSQL's scalability and ACID compliance ensure data integrity for high-frequency operations (e.g., saving test results) while supporting complex queries for generating user progress reports.

1.2.4 User Authentication

KeyDash implements secure user authentication via Supabase Auth, offering email/password sign-up and OTP verification. This ensures only authenticated users can save and compare their typing test results across sessions. Role-based access control (RBAC) restricts sensitive operations, such as modifying test parameters or deleting accounts.

1.2.5 Cloud Storage

Supabase storage manages images given by users for profile setting. Images are stored securely within supabase storage.

1.2.6 Data Modeling

Data modeling involves designing the structure of a database to ensure efficient storage and retrieval of information. In KeyDash, a relational data model was created to map best wpm, id, scores, display name, accuracy, etc.

1.2.7 State Management in React

KeyDash uses React's hooks to manage global state, including:

- Real-time test metrics (keystrokes, errors).
- User authentication status.

1.2.8 Version Control with GitHub

KeyDash uses Git for local version control and GitHub as the remote repository to enable collaborative development. KeyDash uses Git for local version control and GitHub as the remote repository to enable collaborative development. The project leverages CI/CD (Continuous Integration and Continuous Deployment) pipelines to automate testing and deployment workflows with Vercel, ensuring code reliability and efficient updates.

1.2.9 Security Considerations

Security web application is critical, especially when handling user-generated content. KeyDash employs several security practices including authentication checks before file uploads, secure storage of profile images and table policy. By leveraging Supabase's built-in security features, the application maintains a high level of trust and data integrity among its users.

1.3 Project Category (web application)

KeyDash is developed as a web application to provide universal accessibility across devices, including desktops, laptops, and tablets, without requiring platform-specific installations. Unlike mobile apps, a web-based approach ensures instant access via browsers, making it ideal for users who frequently switch between devices (e.g., home computers, library workstations, or shared devices).

The goal of KeyDash is to offer a seamless typing practice experience with real-time analytics, leaderboards, and progress tracking. By choosing a responsive web design, we prioritize:

- **Cross-Platform Compatibility:** Works on any device with a modern browser (Chrome, Firefox, Edge).
- **Ease of Updates:** No app store approvals needed; users always access the latest version.
- **Performance:** Leverages React's virtual DOM for smooth rendering of typing tests and animations.

Built with React.js and Supabase, KeyDash delivers:

- Zero Setup: Users can start typing tests immediately—no downloads or installations.
- Cloud Sync: Progress and scores are saved securely and sync across devices via Supabase backend.
- Keyboard-First UX: Optimized for physical keyboards (common on desktops/laptops), enhancing typing accuracy measurements.

Choosing a web application wasn't just a technical decision—it aligns with how typists and professionals actually practice. Many users prefer full-sized keyboards for training, and web apps eliminate friction for quick practice sessions during work breaks or study time. By making KeyDash browser-based, we meet users where they already are: online, logged in, and ready to improve their typing skills.

1.4 Detailed Problem Definition

The following are the main obstacles to creating a useful typing speed tester:

- Real-time, accurate evaluation of typing speed (WPM) and error rate.
- Designing a user interface that is both visually appealing and intuitive.
- Implementing a system to store and retrieve user performance history.
- Ensuring responsiveness and cross-device compatibility.

KeyDash aims to solve these challenges by utilizing modern web technologies to create a reliable, engaging, and scalable typing assessment tool.

1.5 Presently Available System for the Same Purpose

Currently, users looking to practice or evaluate their typing skills must rely on various disconnected solutions, each with significant limitations. While websites like 10FastFingers offer basic typing tests, they lack comprehensive features such as personalized progress tracking, customizable test options, or user accounts to save results across devices. Desktop applications like TypingMaster provide structured lessons but require installations and often miss real-time feedback and competitive elements. Browser-based tools and mobile apps focus on specific aspects like

algorithmic text generation but fail to integrate social features or centralized performance tracking. Informal methods, such as practicing in document editors, offer no measurable metrics, while gaming platforms prioritize entertainment over skill development. These fragmented solutions share common drawbacks: they don't combine speed testing with accuracy analytics and social competition, provide poor organization of results, and offer minimal customization options. KeyDash addresses these gaps by delivering a unified web platform with secure accounts, real-time analytics, and shareable leaderboards, creating a more complete typing practice solution.

1.6 Objective of the Project

- To extend the typing speed tester into multimode practice mode covering language (Nepali/English), Coding, symbols and Custom text.
- To build habit forming and competitive mechanism: daily streaks, tiered achievements, real time multiplayer races, AI battles and multi-period leaderboards.

1.7 Scope

The scope of KeyDash depends on the successful implementation of an intuitive UI, engaging UX, and features that cater to both casual typists and professionals. Like many popular tools, its effectiveness lies not in complex technology.

The scope of the KeyDash application is:

- KeyDash provides real-time typing tests across Language, Coding, symbols, Custom text modes, with adjustable difficulty, to assess accuracy, speed (WPM), and errors. Complete typing history with replay, streak tracking and 1,200+ tiered achievements give users a detailed picture of their progress over time.
- KeyDash includes daily, weekly, monthly and all-time leaderboards, real-time multiplayer typing races over Supabase Realtime, and AI opponents at four difficulty levels, along with shareable result cards for social platforms , fostering motivation and community engagement.

Chapter 2: Literature Review

2.1 Introduction

Typing speed and accuracy are critical skills in the digital era, impacting productivity and efficiency in academic and professional settings. Research indicates that real-time feedback and structured practice significantly enhance typing proficiency by reducing cognitive load and reinforcing muscle memory [1], [2]. Modern typing tutors leverage interactive methods to improve engagement and learning outcomes [5].

2.2 Content Review

Existing typing test platforms, such as 10FastFingers and TypingClub, offer standardized tests with performance metrics like Words Per Minute (WPM) and accuracy [3], [4]. However, many lack customization options, localized language support, or detailed progress tracking, limiting their effectiveness for structured learning [6].

2.3 Traditional Review

Students and professionals often rely on basic typing tests or manual practice methods, which lack real-time analytics and personalized feedback [6]. While some institutions integrate typing exercises into coursework, these are rarely systematic or data-driven [6]. Additionally, many existing solutions do not support multi-language input or fail to store historical performance data, making long-term progress tracking difficult [6].

2.4 Our Proposed System

KeyDash addresses these gaps by offering a modern, adaptive typing speed tester with real-time WPM and accuracy tracking. Unlike generic platforms, KeyDash supports customizable test durations, multiple input languages, and supabase-backed user profiles for progress monitoring. Built with HTML, CSS, and JavaScript, it ensures accessibility across devices while providing a responsive, user-friendly interface. By focusing on structured practice and performance analytics, KeyDash bridges the divide between casual typing tests and skill-based mastery [7].

Chapter 3: Analysis

3.1 Project Management

The successful creation of KeyDash, a typing speed tester intended to assist users in increasing their typing efficiency, depended heavily on efficient project management. The project lifecycle is described in this part, including requirement analysis, cost-benefit analysis, feasibility analysis, and early research.

3.1.1 Initial Investigation

The idea for KeyDash emerged from recognizing the need for a modern, user-friendly typing speed tester that provides real-time feedback and progress tracking. Existing solutions often lacked customization, multi-language support, or detailed analytics. A preliminary survey among students and professionals confirmed a demand for a tool that combines accuracy measurement (WPM), error analysis, and historical performance tracking. Based on these findings, KeyDash was proposed to offer an intuitive, feature-rich typing assessment platform.

3.1.2 Feasibility Analysis

To ensure the project's viability, four key feasibility areas were evaluated:

- **Technical Feasibility**

The project is technically feasible, leveraging HTML, CSS, and JavaScript for frontend development, ensuring cross-platform compatibility. Supabase was chosen for backend services (authentication and data storage) due to its scalability, real-time updates, and ease of integration. These technologies are well-documented and widely used, making development efficient.

- **Economic Feasibility**

The project is economically viable. All tools and frameworks used (Visual Studio Code, Supabase free tier, GitHub Pages for hosting) are free or open-source. Development was conducted on existing hardware, eliminating additional costs. Future expenses, if any, would involve optional Supabase plan upgrades for increased user traffic.

- **Schedule Feasibility**

A clear development timeline was set and followed throughout the project. Over the course of several weeks, the complete system was created, including stages such as planning, designing, coding, testing, and final deployment. The project was time-feasible because it was finished inside the academic calendar.

- **Legal Feasibility**

The application does not violate any intellectual property or institutional regulations. It is intended for educational use, and all materials shared on the platform are user-uploaded. The system includes privacy-respecting authentication and does not collect or misuse sensitive personal data.

3.1.3 Cost Benefit Analysis

The project's costs were minimal:

- Development tools: Free (VS Code, Supabase, GitHub).
- Testing: Conducted on personal devices.
- Deployment: Free via Vercel.
- Documentation: Physical copy (10 per page costs).

3.2 Requirement Analysis

To ensure KeyDash is functional, secure, and user-centric, the following requirements were prioritized:

- **Eligibility**

Only authenticated users (via email/password or Google login) can save progress. This prevents misuse and ensures personalized tracking.

- **Uniqueness**

Each test session produces a unique result that is linked to the user's account. This stops duplicate entries and lets for accurate progress analysis.

- **Privacy**

Supabase securely stores and encrypts user data, including email and WPM

history. There is no implementation of permanent text storage or keystroke logging.

- **Transparency**

Users can view their historical performance but are not tracked beyond aggregated metrics (WPM, accuracy). No intrusive data collection is performed.

Chapter 4: Design

4.1 Software Requirements Specification

KeyDash was developed using industry-standard tools including Visual Studio Code as the primary IDE for frontend coding (HTML/CSS/JavaScript), Figma for UI/UX design prototyping, Supabase for backend authentication and real-time data management, and GitHub Pages for free deployment. These technologies were carefully chosen for their accessibility, seamless integration capabilities, and developer-friendly features to optimize the development workflow and ensure project success.

4.1.1 Functional Requirements

The functional requirements of KeyDash are:

- **User Authentication:** The system shall allow users to register and log in via email/password or Google authentication.
- **Real-Time Typing Test:** The app shall measure typing speed (WPM), accuracy, and error count in real time.
- **Test Customization:** Users shall select test duration (30s, 60s, 120s) and text source (random words, quotes, or custom input).
- **Progress Tracking:** Authenticated users shall view their historical test results, including WPM trends and accuracy improvements.
- **Responsive Design:** The interface shall adapt seamlessly to desktops, tablets, and mobile devices.

4.1.2 System Specification

1. Software Requirements

- Browser
- Prototyping: Figma
- Logo design: Adobe illustrator
- User Interface: React
- Back End: Supabase

- Database: PostgreSQL

2. Hardware Requirements

- Device that supports web service.

4.1.3 Non-functional Requirements

The following are some non-functional requirements of our system:

- Performance: The app shall load quickly (<2s) and run smoothly, even on low-end devices.
- Usability: The interface shall be intuitive, with minimal navigation steps to start a test or view results.
- Scalability: Supabase backend shall handle increasing users and data without performance degradation.
- Security: User data (email, test history) shall be encrypted and stored securely via Supabase Authentication.
- Maintainability: Modular JavaScript code shall allow easy updates and feature additions.
- Portability: The web app shall function across browsers (Chrome, Firefox, Edge) and devices.
- Reliability: Real-time Supabase synchronization shall ensure data consistency, with graceful error handling.

4.2 Risk Assessment

Developing a typing speed tester like KeyDash involves technical, operational, and user-related risks that must be proactively managed.

Technical Risks:

A major concern is server dependency, as KeyDash relies on Supabase for authentication and data storage. Any Supabase service disruption could temporarily disable user logins or progress tracking. Another risk is inaccurate WPM/accuracy calculations, which could undermine user trust. Rigorous algorithm testing and real-time validation checks ensure consistent performance metrics.

Security & Data Risks:

Unauthorized access to user profiles or test history could compromise privacy. Supabase's built-in authentication and encryption minimize this risk, while strict access controls prevent data leaks. Data loss from accidental deletions or sync failures is mitigated through Supabase's automatic backups and versioning.

User Adoption & Behavior:

Low engagement may occur if users find the interface unintuitive or lack motivation to track progress. KeyDash counters this with a gamified design (e.g., progress charts, achievement badges) and seamless onboarding.

Accessibility & Compatibility:

Disparities in device performance or browser support could create inconsistent experiences. The app is rigorously tested across browsers (Chrome, Firefox, Edge) and screen sizes, with fallback options for older devices. Offline functionality allows practice sessions without internet, though analytics require reconnection.

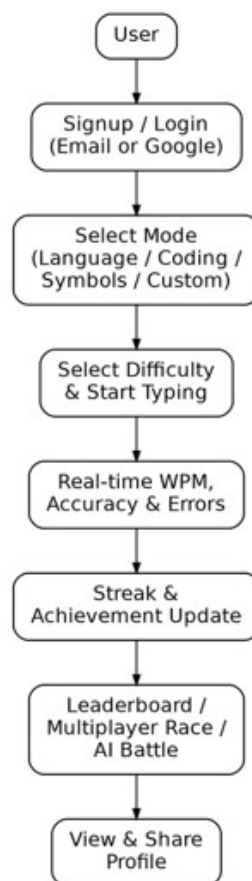
By addressing these risks through robust architecture, user-centric design, and fail-safes, KeyDash ensures reliability and long-term usability for typists at all skill levels.

Chapter 5: System Modeling

5.1 Block Diagram

In block diagram for android application, it works as follows

- i. User signup with the email and password.
- ii. User select practice mode: language, Symbols, Custom, Coding.
- iii. User select difficulty and start typing.
- iv. System updates the user's daily streak and achievement update.
- v. User view leaderboard and join multiplayer's race and battles an AI opponent.
- vi. User view & manage profile and can share result cards on social media.



5.2 UML Diagram

5.2.1 UML Use Case Diagram

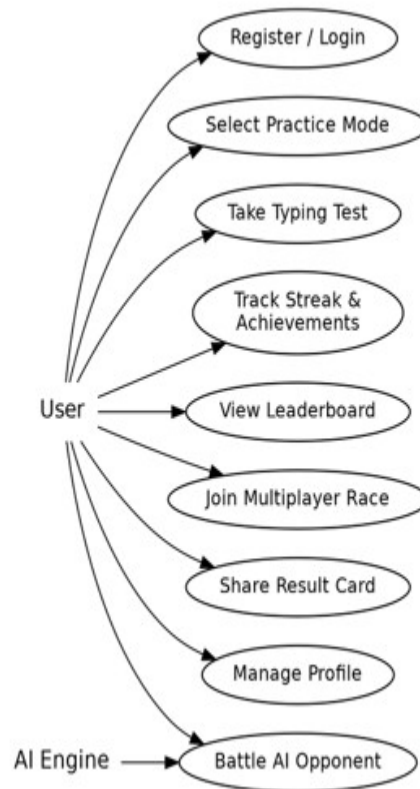
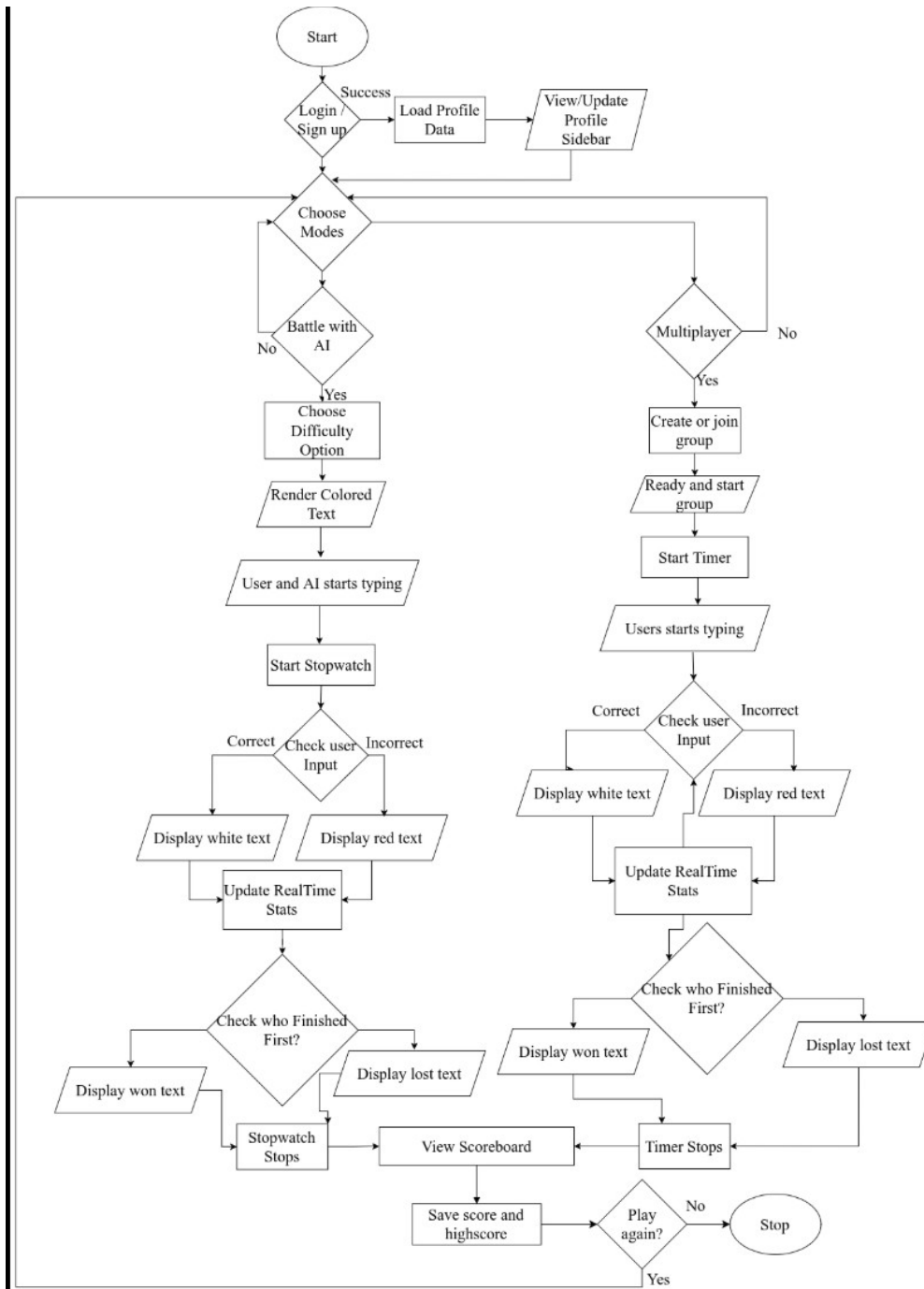


Figure 5.2: UML Use Case Diagram

5.3 Flowcharts



System Flowchart

The flowchart of KeyDash illustrates the user interaction flow in the typing speed tester, starting with login/signup authentication, followed by profile data loading and mode selection (Timed or Sentence). In Sentence Mode, the user selects a difficulty

level, and the system renders colored text for typing; correct inputs turn text white, while errors turn it red, with real-time stats updating dynamically. The process continues until the user completes the sentence, triggering score.

5.4 System Workflow

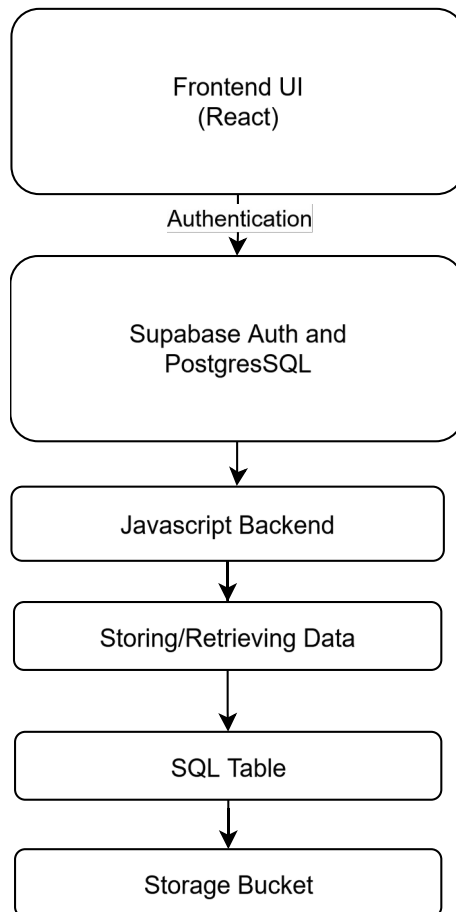


Figure 5.4: System Workflow

5.4.1 Registering Users

User with a valid email address is capable of registering in the system. Supabase provides its own authentication procedure to make sure whether the email can be registered or not. It sends the confirmation link in mail to complete the registration procedure. After following the confirmation link it directly log in to the system.

5.4.2 Log In Option

User with the confirmed email is able to login to the system. If the valid email is entered only then the user is able to gain access in the system, if email is not confirmed then user is prompted to confirm the email. Our system provides passwordless login. User are also provided with the option of continuing with their google account i.e. Gmail account.

5.4.3 View modes and respective levels

User after logging into the system and choose the level they want or refers. User can get access to the difficulty modes even if they haven't logged in the system, it is not mandatory. But without it they cannot record or track their progress.

5.4.4 View Leaderboard

The authenticated users can look up to the leaderboard of the modes and compete to other users in different modes with their respective levels. Every mode have their own leaderboard where the top score are mentioned on the basis of key metrics (WPM, accuracy, etc.).

5.4.5 View Scoreboard

After user completes typing in the selected mode, a scoreboard appears. It includes WPM, accuracy, mistakes, characters, etc. User can view it after they completes their task.

5.4.6 Profile Section

On the profile section, users are able to track back their all of the past session. They can view their best score, average accuracy and total number of sessions with their record.

5.4.7 Multiplayer Race

Authenticated users can create or join a race room and type against other live users. Supabase Realtime broadcasts each participant's progress so every racer's position updates instantly on everyone's screen, and the winner is determined by whoever finishes with the highest WPM and accuracy.

5.4.8 AI Battles

Users who prefer to practice alone can instead challenge a simulated AI opponent at Easy, Medium, Hard, or Pro difficulty. The AI opponent types at a target WPM appropriate to the chosen difficulty, giving the user a consistent benchmark to try to beat without needing another live player.

5.4.9 Profile Section

On the profile section, users are able to track back their all of the past session. They can view their best score, average accuracy and total number of sessions with their record.

5.5 Data Flow Diagram

5.5.1 Context Level DFD

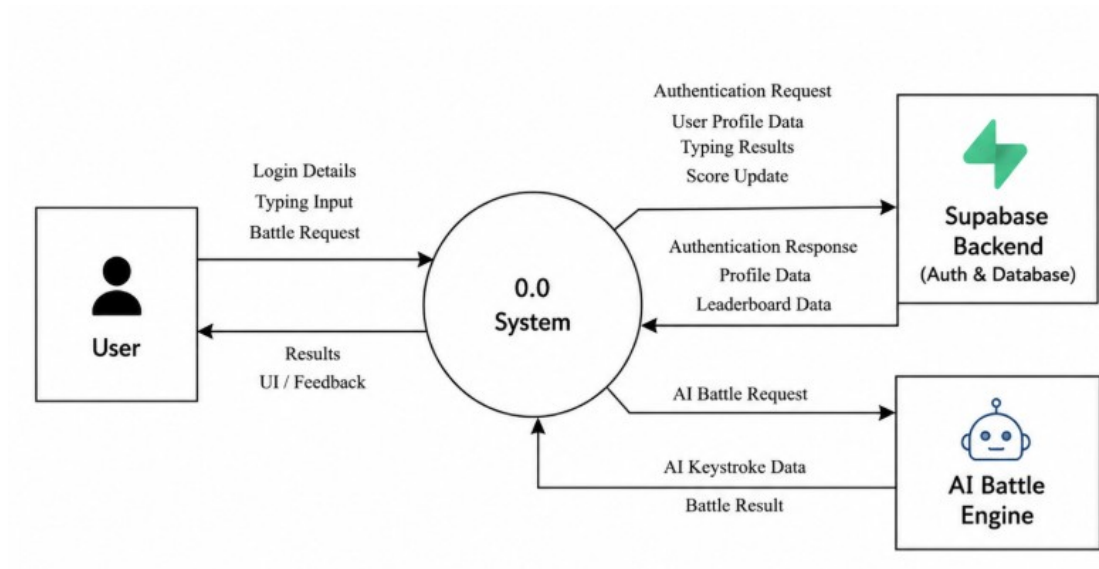


Figure 5.5: Context Level DFD

5.5.2 Level-1 DFD

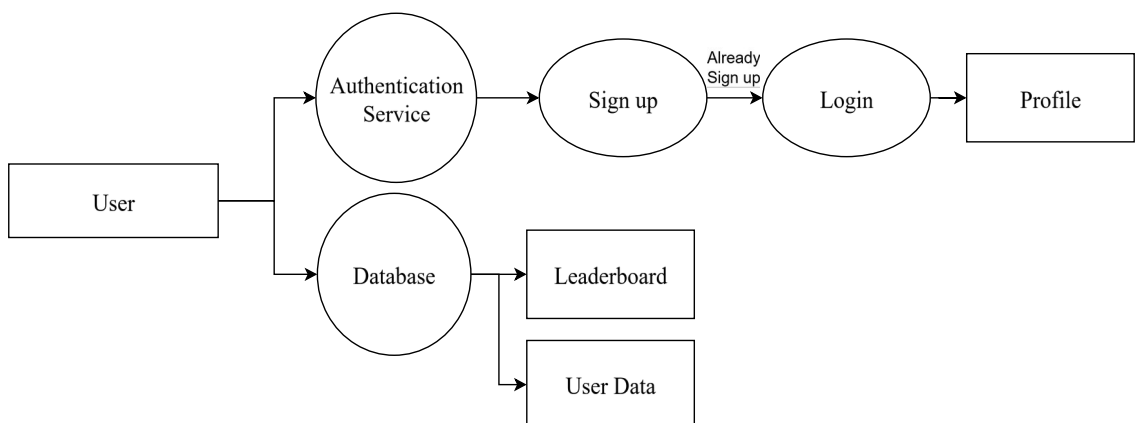


Figure 5.6: Level-1 DFD

5.6 Database Design

5.6.1 ER (Entity Relationship) Diagram

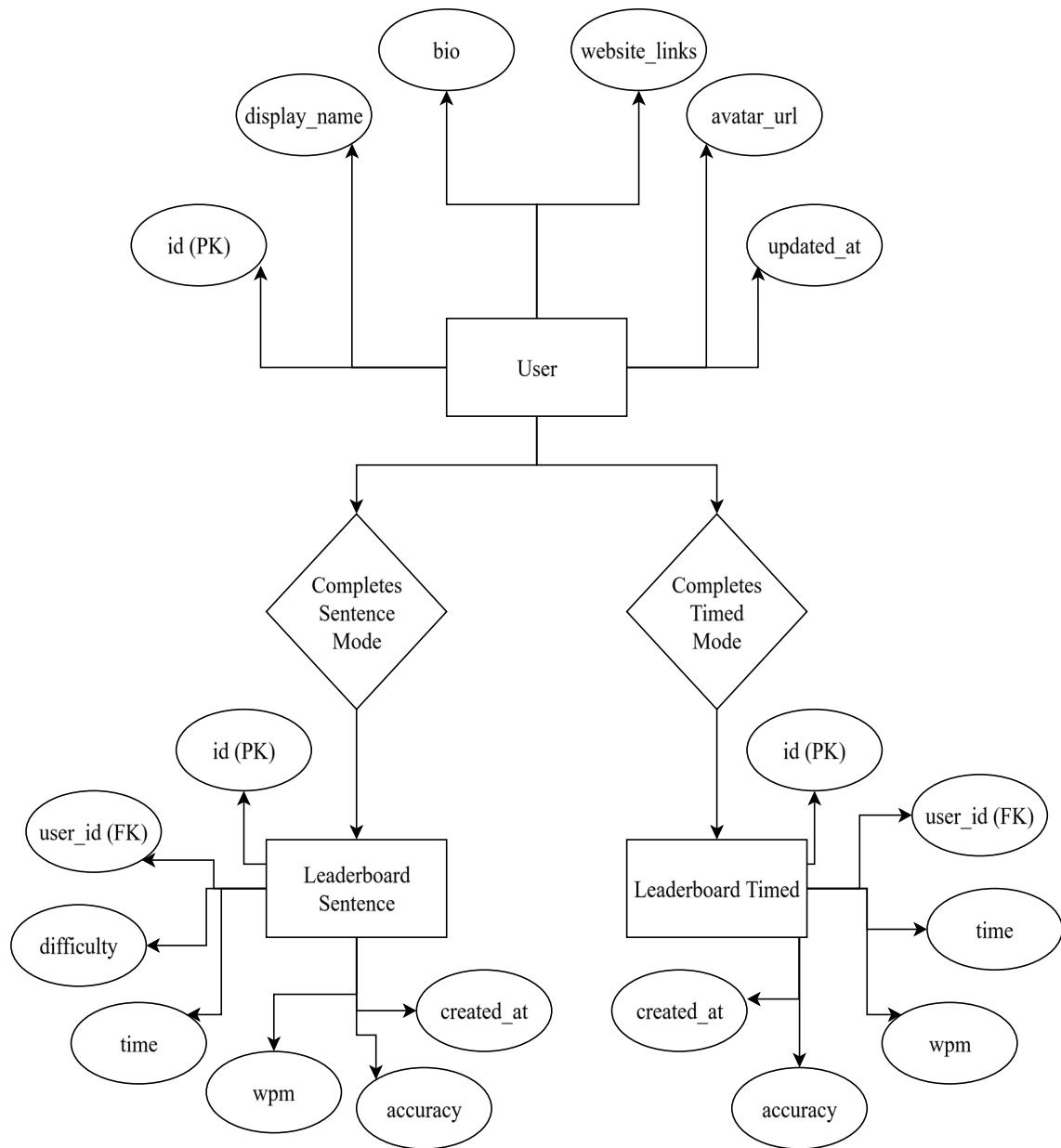


Figure 5.7: ER Diagram

5.6.2 Supabase Database Table for users

Supabase Database table basically contain 3 table: profile, leaderboard_sentence and leaderboard_timed.

S.N.	Field	Data Type	Description
1	id	UUID	Id of the user identifier
2	display_name	Text	Displays the user name
3	bio	Text	Display the user biodata
4	twitter	Text	Twitter info
5	github	Text	GitHub info
6	linkedin	Text	LinkedIn info
7	instagram	Text	Instagram info
8	created_at	Time Stamp	Timestamp when user data was recorded
9	total_tests	Integer	Records total tests taken
10	best_wpm	Integer	Records the highest wpm
11	average_wpm	Numeric	Records the average wpm

Table 5.1: Supabase Table Profile

S.N.	Field	Data Type	Description
1	id	UUID	Id of the user identifier
2	user_id	UUID	Id of the user
3	difficulty	Text	Difficulty level of sentence mode
4	time	Numeric	Time taken to complete the entire sentence
5	wpm	Numeric	Word per meter
6	accuracy	Numeric	Accuracy of typing
7	created_at	Time Stamp	Timestamp when user data was recorded

Table 5.2: Supabase Table Leaderboard_sentence

S.N.	Field	Data Type	Description
1	id	UUID	Id of the user identifier
2	user_id	UUID	Id of the user
4	time	Numeric	Time option is provided
5	wpm	Numeric	Word per minute
6	accuracy	Numeric	Accuracy of typing
7	created_at	Time Stamp	Timestamp when user data was recorded

Table 5.3: Supabase Table Leaderboard_timed

5.6.3 Supabase As a Database

Supabase was selected as the backend solution for KeyDash due to its simplicity, flexibility, and suitability for real-time applications. Supabase is an open-source Backend-as-a-Service (BaaS) that provides a complete backend ecosystem built on top of PostgreSQL. It includes features such as authentication, real-time database, file storage, etc making it ideal for a dynamic typing speed tester like KeyDash.

A key advantage of Supabase is its JavaScript/React SDK, which enables straightforward integration with frontend frameworks. This allows for efficient implementation of user authentication, data storage, and real-time updates without

complex server management. Built on PostgreSQL, Supabase ensures high performance, scalability, and relational database capabilities, essential for managing user profiles, typing statistics, and leaderboard rankings.

In the context of KeyDash, Supabase is used for:

- Authentication: Secure user registration, login, and session management via email/password.
- Database: Stores user profiles (name, email), per-mode test results, per-mode leaderboard entries, streak counters, unlocked achievement, multiplayer room/participants record, and typing performance metrics (WPM, accuracy, test history).
- Real-time Updates: Instantly syncs leaderboard data and personal best scores across all users.

Supabase's user-friendly dashboard simplifies database schema management, query execution, and real-time monitoring—accelerating development and debugging. KeyDash's use of Supabase results in a low-maintenance, scalable, and secure backend that improves user experience and developer productivity.

Chapter 6: Coding

6.1 Hardware Specification

Since KeyDash is a web-based application, it does not require high-end hardware specifications. It can run in any device that support web services.

6.2 Platform

KeyDash is developed as a web-based application, built using modern web technologies to ensure cross-platform compatibility and seamless accessibility. Unlike native mobile apps, it runs directly in web browsers, eliminating the need for downloads or installations while providing full functionality across desktops, laptops, and tablets. The application leverages React.js for its responsive frontend and Supabase for backend services, delivering real-time typing analytics, user authentication, and cloud-based data synchronization. As a web app, KeyDash prioritizes universal access, allowing users to practice typing anytime, anywhere, without device restrictions—simply by visiting the platform through a browser. This approach combines the flexibility of web applications with the performance of native-like experiences, making it an ideal solution for typing enthusiasts, students, and professionals alike.

6.3 Technology Used

6.3.1 React Technology

React is an open-source JavaScript library developed by Facebook for building dynamic and responsive user interfaces. In the development of KeyDash, React was chosen as the primary frontend technology due to its component-based architecture, efficient rendering with the Virtual DOM, and strong ecosystem of tools and libraries.

React uses JavaScript (JSX) for declarative UI development, allowing seamless integration of HTML-like syntax within JavaScript. This approach enables developers to create reusable UI components, making the codebase modular and easier to maintain. For KeyDash, this meant efficiently rendering real-time typing metrics, such as WPM (Words Per Minute) and accuracy, with smooth updates.

One of React most powerful features is state management, which allows dynamic data handling without full page reloads. Using React Hooks (`useState`, `useEffect`), KeyDash tracks user input, calculates performance metrics, and updates the UI instantly. Additionally, React unidirectional data flow ensures predictable behavior, making debugging and testing more straightforward.

For styling, CSS-in-JS libraries (such as Tailwind CSS) were used to create a responsive and visually appealing interface. React compatibility with Supabase also enabled seamless integration for user authentication and data storage, allowing users to save and track their typing progress over time.

Overall, React proved to be an excellent choice for KeyDash, offering high performance, cross-platform compatibility (web and mobile browsers), and a streamlined development process making it ideal for a real-time, interactive typing speed tester.

6.4 Programming Language Used

6.4.1 JavaScript Programming Language

JavaScript is a versatile, high-level programming language that serves as the core technology for web development. As the primary language for KeyDash, JavaScript enables dynamic and interactive web functionality, making it ideal for real-time applications like typing speed tests.

JavaScript operates within the browser's runtime environment, allowing immediate execution of code without compilation. Its event-driven architecture supports asynchronous operations, which is crucial for KeyDash to handle user inputs, calculate typing metrics (WPM, accuracy), and update the UI seamlessly.

KeyDash harnesses several powerful JavaScript features to deliver optimal performance, including DOM manipulation for real-time webpage updates that display typing results instantly, seamless React integration that enables component-based UI development through JavaScript's foundational support, and robust JSON handling capabilities that facilitate smooth data processing for Supabase interactions and local storage management.

6.4.2 SQL (Structured Query Language)

SQL is the standard language for managing relational databases. In KeyDash, SQL powers data operations through Supabase, a PostgreSQL-based platform that offers full SQL support while adding modern features like real-time subscriptions and auto-generated APIs.

KeyDash leverages core SQL operations through Supabase PostgreSQL backend, including data retrieval of user history (such as previous WPM scores) using SELECT queries, profile and test result modifications via UPDATE commands, and performance trend analysis utilizing PostgreSQL's powerful analytical functions for comprehensive typing insights.

Unlike NoSQL alternatives, Supabase leverages PostgreSQL's strict SQL compliance, enabling complex queries with joins, transactions, and referential integrity – crucial for features like leaderboards and progress tracking.

6.5 Software Tools Used

6.5.1 Visual Studio Code

Visual Studio Code (VS Code) is a lightweight yet powerful source code editor developed by Microsoft, used as the primary development environment for KeyDash. Its built-in support for JavaScript, React, and SQL (via extensions) made it ideal for implementing the typing tester's frontend logic and database interactions. Features like IntelliSense for code completion, integrated terminal, and Git version control streamlined the development process, while extensions like ESLint and Prettier maintained code quality. The debugger tool was particularly useful for testing real-time typing analytics before deployment.

6.5.2 Git and GitHub

Git served as the version control system for tracking all code changes in KeyDash, while GitHub hosted the repository remotely. This combination enabled seamless collaboration—branching prevented conflicts during feature development (e.g., Supabase integration), and pull requests ensured code reviews before merging.

GitHub Issues helped organize tasks like UI improvements and bug fixes, facilitated demo deployments during testing phases.

6.5.3 Web browser

Modern browsers like Chrome and Edge were essential for testing KeyDash's responsiveness and real-time features. Their DevTools allowed debugging JavaScript performance, inspecting DOM updates during typing tests, and auditing Lighthouse metrics for optimization (e.g., WPM calculation speed). Browser extensions like React Developer Tools further simplified UI component inspection.

6.5.4 Adobe Illustrator

Adobe Illustrator was used to design KeyDash's logo and custom UI assets (e.g., animated progress indicators). Its vector-based workflow ensured graphics remained sharp across devices, while SVG exports integrated smoothly with React for dynamic theming.

6.5.5 Vercel

My React + Vite website was deployed using Vercel, a technology renowned for its ease of usage and performance. Every time we upload improvements to GitHub, Vercel builds and launches our website immediately. We were able to test features using live preview URLs and iterate more quickly thanks to our continuous deployment strategy. Only the build command, output directory, and required .env file needed to be specified for Vercel's smooth support of Vite.

6.6 Coding Style Followed

KeyDash follows a component-based architecture using React, which promotes modularity and reusability in the codebase. The UI is broken down into independent components each managing its own state and logic. This approach ensures clean separation of concerns and simplifies debugging. Event-driven programming handles real-time user interactions, such as keystroke detection and WPM calculation, with React hooks managing state updates efficiently.

For backend interactions with Supabase, functional programming principles are applied, emphasizing pure functions and predictable data flow. SQL queries are structured in dedicated service modules, keeping database logic separate from UI components. Consistent code formatting enforces readability, while descriptive variable names (e.g., `wpm` instead of `x`). This hybrid style combining React's declarative UI patterns with functional data handling ensures a scalable and maintainable typing tester.

Chapter 7: Testing

7.1 Test Plan

Manual testing is inefficient and doesn't scale. Test designs are used for reuse which improve our application efficiency. So, we have done unit test, integration testing, black-box and white-box testing and system testing as a whole.

7.2 Test Cases

7.2.1 Testing Cases Summary

By looking at the characteristics of developed smart phone application, it tries to understand the factors that lead to a note uploading system, but also figures out the problem that may lead to the failure we have done different test our application project and the result of upgrading the application for future prospects to improve the application.

Table 7.1 : Test case Summary

S.N.	Test Case Summary	Results
1	To test whether it displays differently to incorrect character.	Passed
2	To Restart the dashboard for next display text	Passed
3	To test whether it displays score, graph and leaderboard.	Passed
4	To Verify real-time synchronization in multiplayer races.	Passed
5	To Verify multi-language switch (Nepali/English)	Passed
6	To test whether it keeps the record of the user progress or not.	Passed
7	Test AI opponent behavior at different difficulty levels.	Passed

7.2.2 Sample Test Cases

Table 7.2 Test Case 1

Test Cases	1
Test Objective	To test whether it displays differently to incorrect character.
Test Data	In the end, we work remember rao the words of our enemies.
Expected Result	Error should have a font color of red with character being underlined.
Actual Result	The error is underlined and it's font color is red.
Conclusion	Work out is successful.

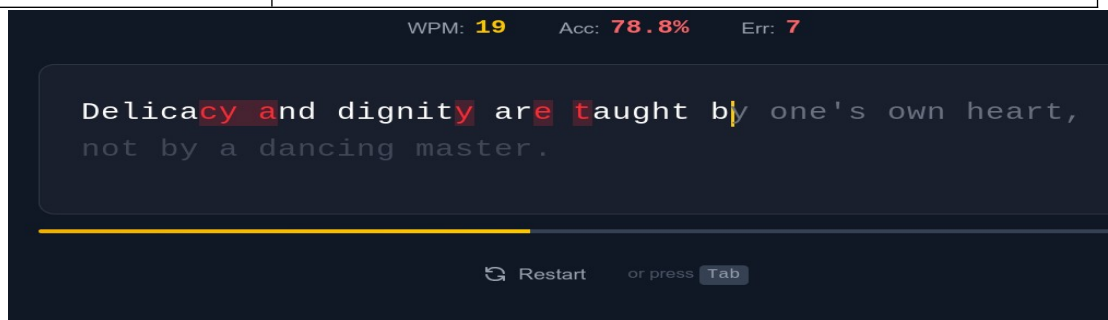


Figure 7.2: Test case 1

Table7.3: Test Case 2

Test Cases	2
Test Objective	To test whether the user can restart the dashboard for the next display text.
Test Data	All he ever wanted out of life was love. You see, he just didn't have any to give
Expected Result	The all 3 metrics (WPM, Accuracy and error) should be displayed correctly according to the data.

Actual Result	The key metrics data is accurately shown.
Conclusion	Work out is successful.

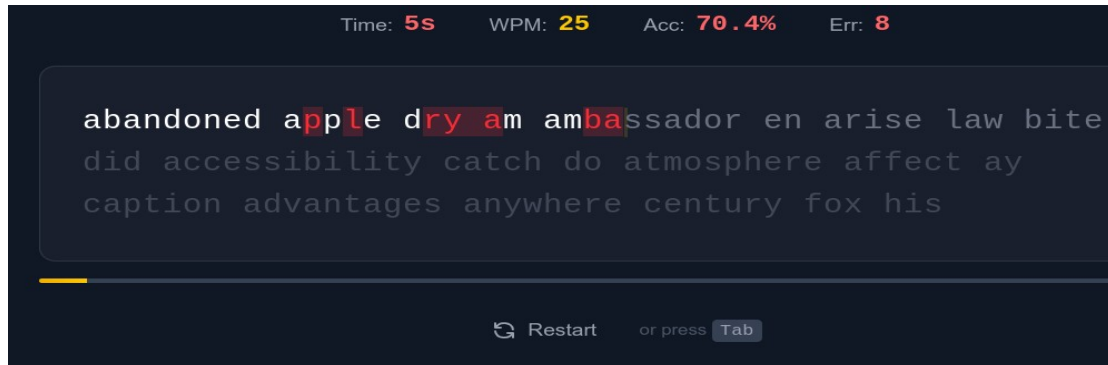


Figure 7.3: Test case 2

Table 7.4: Test Case 3

Test Cases	3
Test Objective	To test whether it displays score and graph and leaderboard.
Expected Result	After completion of session all of key metrics (WPM, accuracy, mistakes, characters, time taken) with the graphical representation of WPM and mistakes should be displayed.
Actual Result	All key metrics with graph is displayed.
Conclusion	Work out is successful.

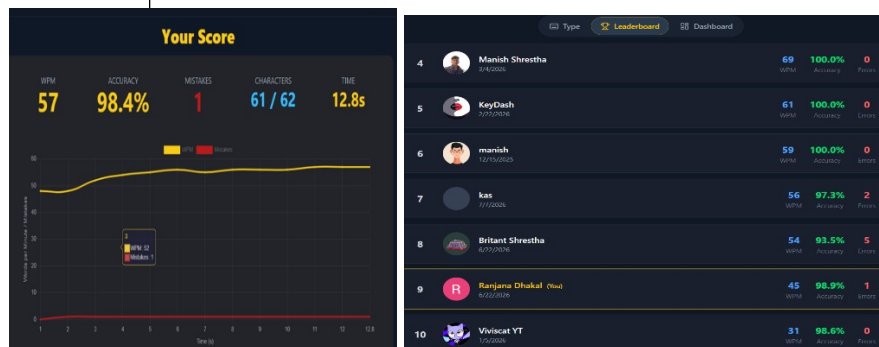


Figure 7.3: Test Case 3

Table 7.5 : Test Case 4

Test Cases	4
Test Objective	To verify real-time keystroke and progress synchronization during a multiplayer typing race.
Expected Result	Competitor's progress indicators should update in real- time with minimal latency using Supabase Realtime broadcasting.
Actual Result	Player progress is broadcasted and updated synchronously.
Conclusion	Work out is successful.

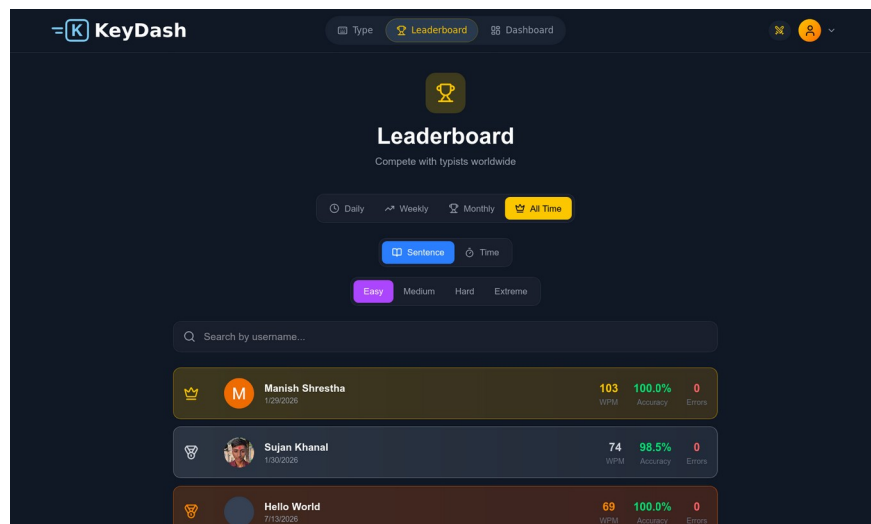


Figure 7.5: Test Case 4

Test Cases	5
Test Objective	Verify multi-language switch(English/Nepali)
Test Data	Ghar farkera Atithi biruwa ropxah.
Expected Result	Selecting the Nepali language option should instantly change the typing test content and UI layout to support Nepali fonts.
Actual Result	The text successfully switches to Nepali .
Conclusion	Work out is successful.

Table 7.6: Test Case 5



Figure 7.6: Test case 5

Table 7.7: Test Case 6

Test Cases	5
Test Objective	To test whether it keeps the record of the user progress or not.
Test Data	Open user profile.
Expected Result	The metrics (WPM and accuracy) with user past session record should be displayed.
Actual Result	The detailed record of past session and user highest WPM and average accuracy is displayed.
Conclusion	Work out is successful.

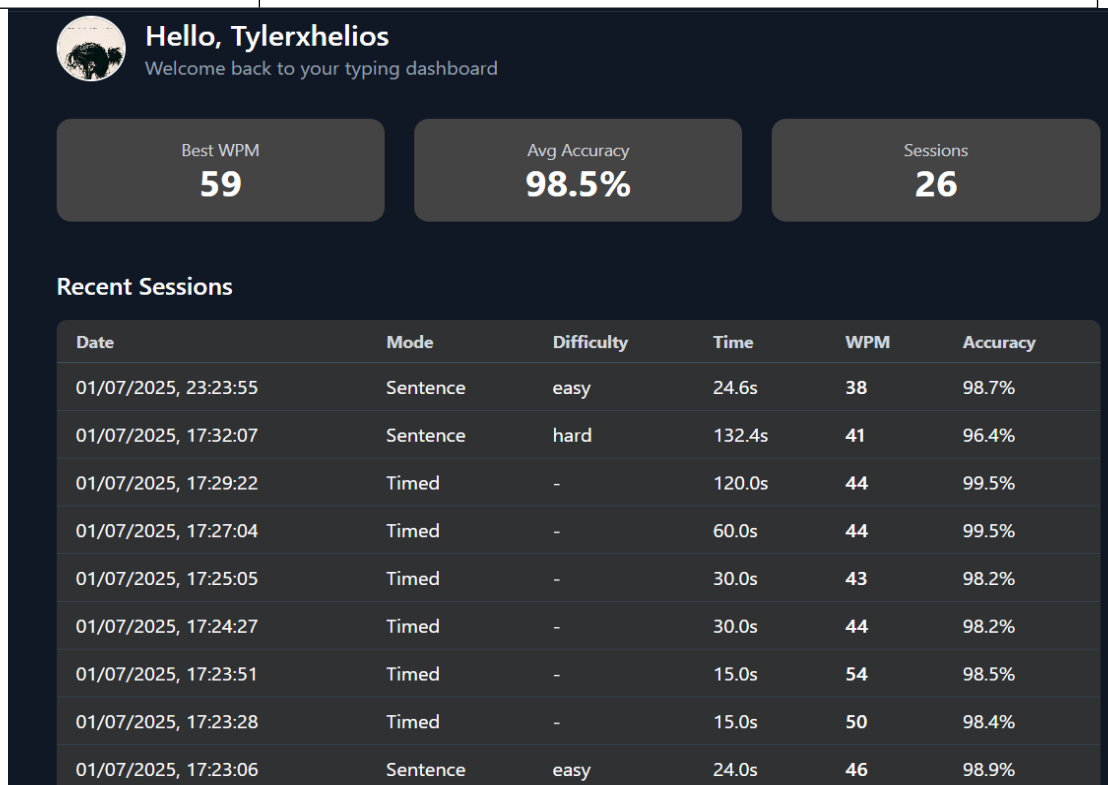


Figure 7.7: Test case 6

Test Cases	6
Test Objective	To test the functionality of AI opponents in an AI battle session.
Test Data	He is our friend
Expected Result	The AI should simulate typing speeds accurately
Actual Result	The typed
Conclusion	Work out is successful.

Table 7.8: Test Case 7

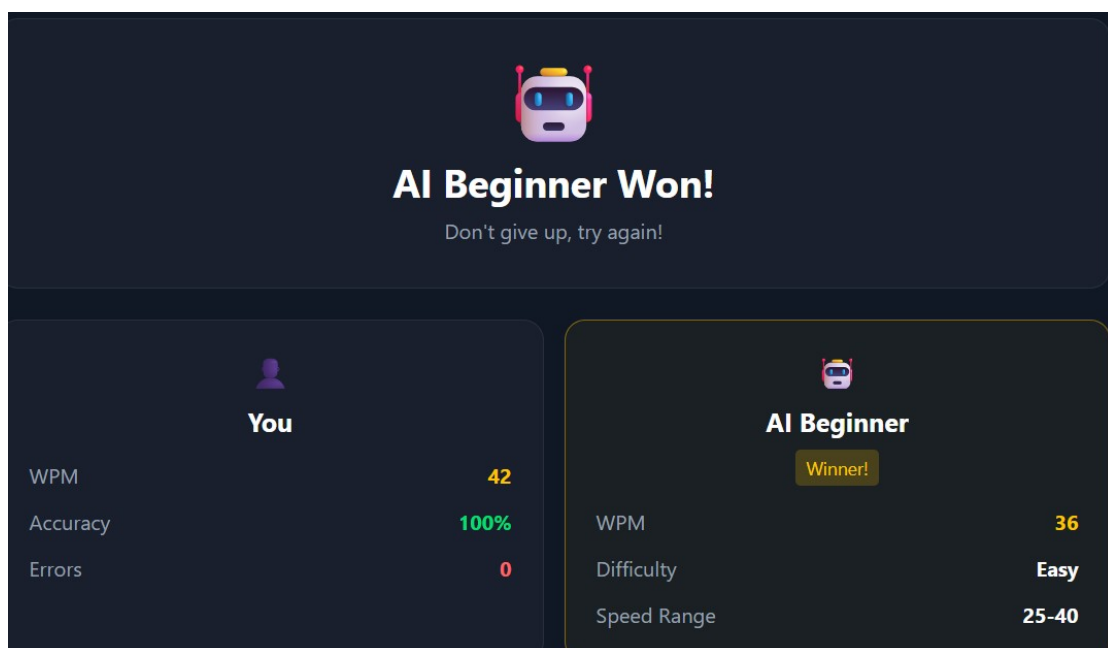


Figure 7.8: Test case 7

Chapter 8: Limitations And Recommendations

8.1 Limitations

However, we attempt to banish all of our encountered errors. Some limitations are:

- KeyDash requires constant internet connectivity for Supabase integration, lacking offline functionality.
- The tool provides only basic metrics (WPM/accuracy) without advanced typing analytics or personalized feedback.
- No While AI battles support four difficulty levels, multiplayer races currently support only small, ad-hoc lobbies rather than large scale matchmaking.
- Mobile testing accuracy may be skewed by on-screen keyboard auto-correction features.
- The Custom mode currently accepts plain pasted text only, without syntax highlighting for less common programming languages beyond the five currently supported.

8.2 Recommendation

These limitations can be solved by further enhancement of the system. We can further make more attractive UI in the future as per the recommendations by the users.

Chapter 9: Conclusion

9.1 Conclusion

KeyDash successfully delivers an efficient and user-friendly typing speed tester that helps users measure and improve their typing proficiency through real-time analytics. By leveraging modern web technologies like React.js and Supabase, the project achieves a responsive, interactive interface with accurate WPM and accuracy calculations. The integration of Supabase ensures secure data storage and retrieval, while the clean UI design enhances user experience across devices. Although the current version has some limitations, such as lack of offline functionality and advanced analytics, it provides a solid foundation for future enhancements.

Overall, KeyDash demonstrates the effective application of full-stack development principles to create a functional and scalable typing assessment tool. The project not only fulfills its core objective of evaluating typing speed but also showcases the potential for expansion with features like multiplayer mode, customized practice texts, and detailed performance insights. As a college project, it highlights the developer's technical competency in building web applications while leaving room for iterative improvements to meet diverse user needs.

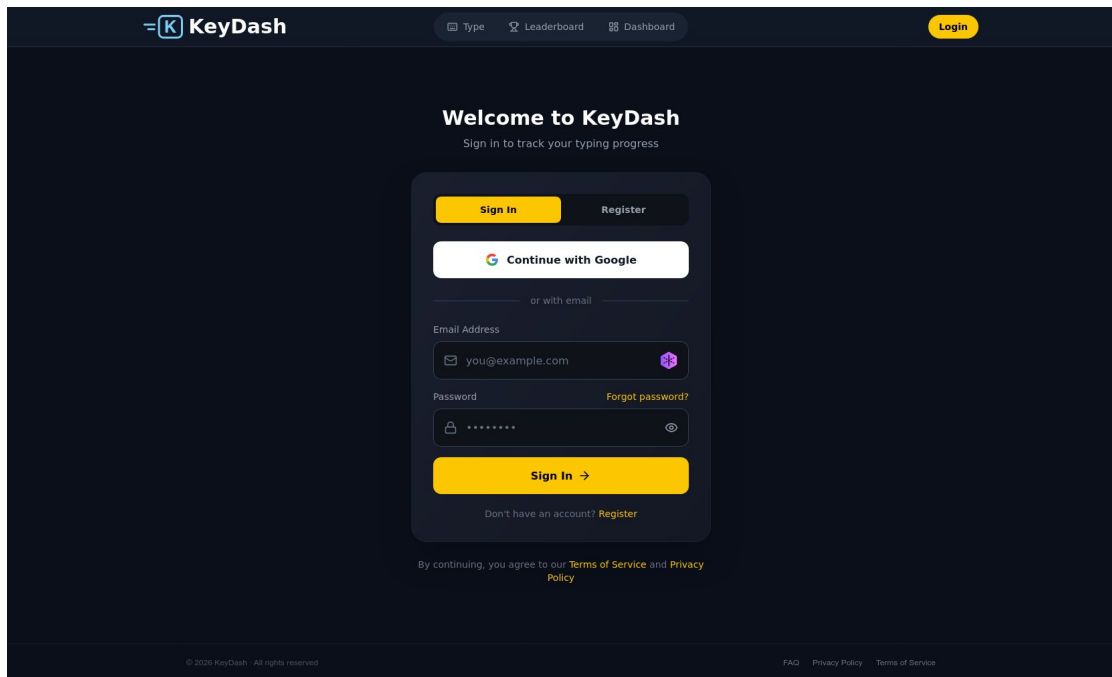
We thank you all who have helped us directly or indirectly in bringing this project to life.

We have tried our best to give you the very best result and hope our users find great value in it.

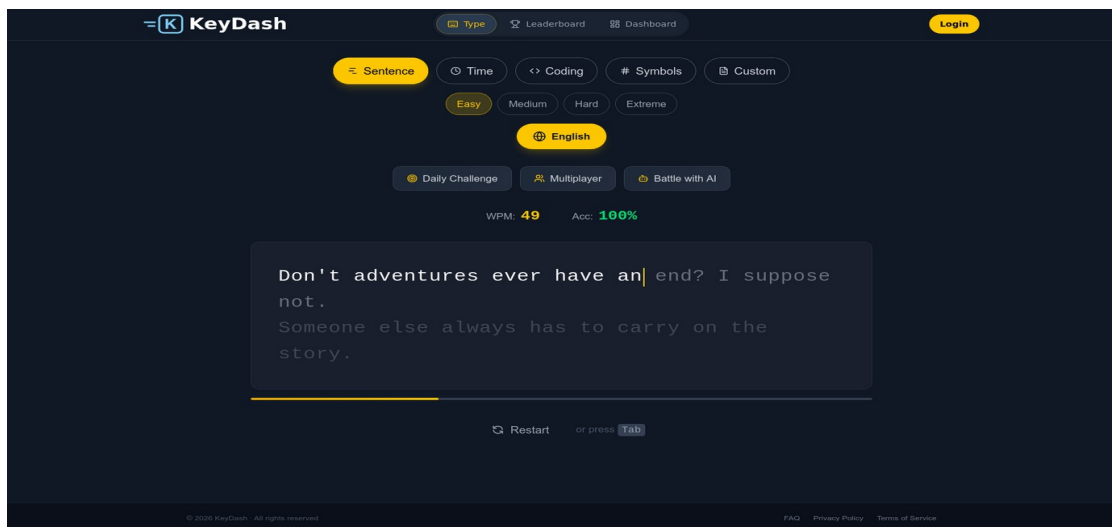
REFERENCES

- [1] D. H. Smith and J. A. Doe, "The Impact of Real-Time Feedback on Typing Proficiency," *Journal of Computer-Assisted Learning*, vol. 12, no. 3, pp. 45–60, 2020.
- [2] R. Brown and M. Lee, "Cognitive Load Theory and Skill Acquisition in Typing," *International Journal of Human-Computer Studies*, vol. 78, pp. 112–125, 2021.
- [3] 10FastFingers, "Typing Speed Test Methodology," 2019. [Online]. Available: <https://www.10fastfingers.com/methodology>
- [4] TypingClub, "Gamified Learning in Typing Education," *EdTech Review*, 2022. [Online]. Available: <https://www.typingclub.com/research>
- [5] A. Taylor et al., "Engagement in Digital Typing Tutors: A Meta-Analysis," *Computers & Education*, vol. 95, pp. 89–102, 2021.
- [6] K. Patel and S. Kim, "Limitations of Current Typing Tools in Structured Learning," *Proc. of the ACM Conference on Learning Systems*, pp. 205–220, 2022.
- [7] L. Garcia, "Adaptive Learning Systems for Typing Mastery," *IEEE Transactions on Education*, vol. 65, no. 4, pp. 550–565, 2023.

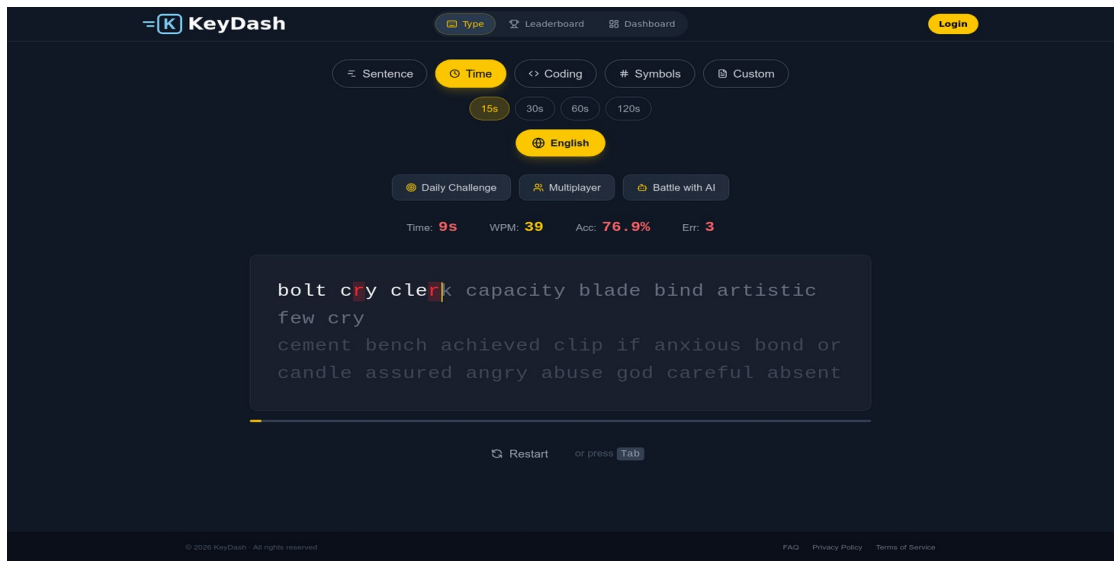
APPENDICES



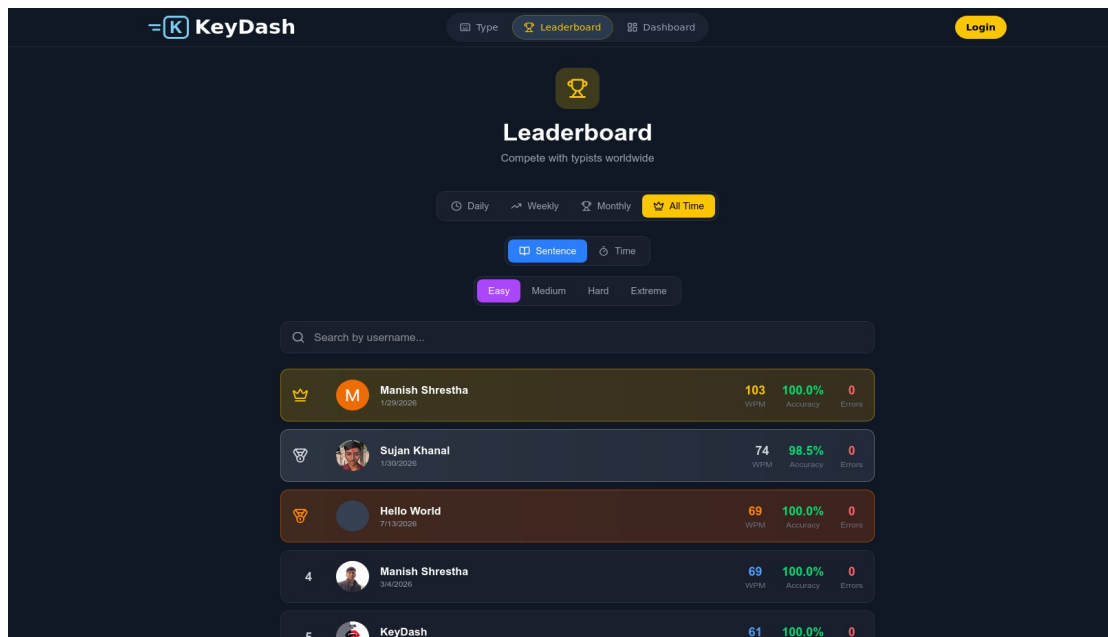
Appendix A: Login Screen



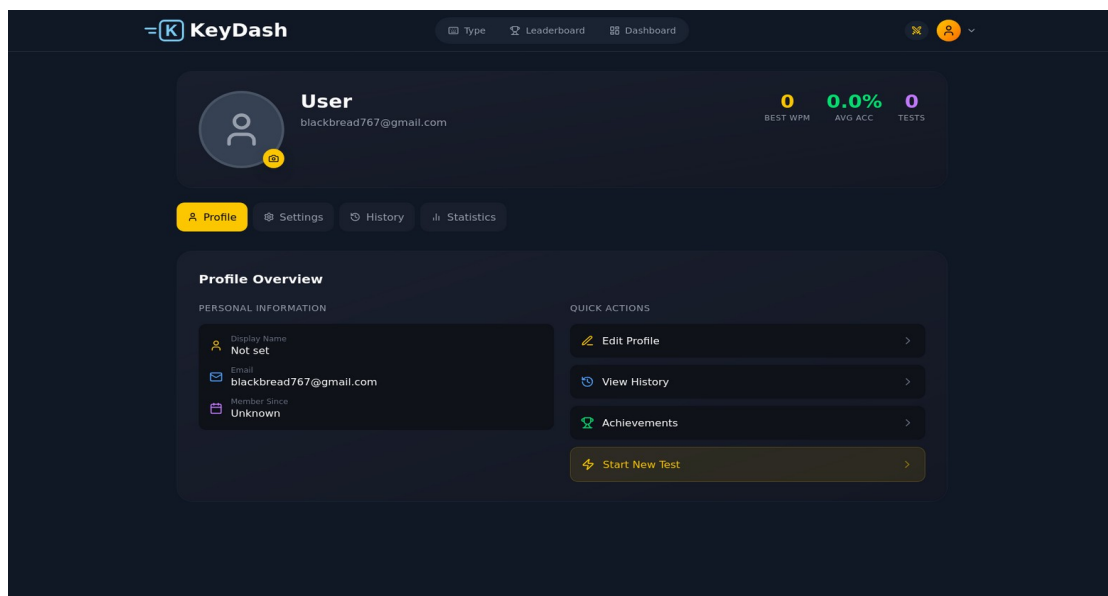
Appendix B: Sentence Mode



Appendix C: Timed Mode



Appendix D: Leaderboard



Appendix E: Profile Section

KeyDash

Type

Leaderboard

Dashboard

User

blackbread767@gmail.com

0

BEST WPM

0.0%

AVG ACC

0

TESTS

Profile

Settings

History

Statistics

Edit Profile

Display Name *

Your display name

Email

blackbread767@gmail.com

Bio

Write a few sentences about yourself...

Website

https://example.com

SOCIAL LINKS

X (Twitter)

username

GitHub

username

LinkedIn

username

Instagram

username

YouTube

channelname

Twitch

username

Reddit

username

Snapshot

username

PUBLIC PROFILE PREVIEW

Profile visibility

If private, other users cannot see your achievements on your public profile.

Public

Featured achievements

No featured achievements selected yet.

0/4 selected

Choose achievements to show on public profile

Unlock achievements first to feature them on your public profile.

Select up to 4

Save Changes

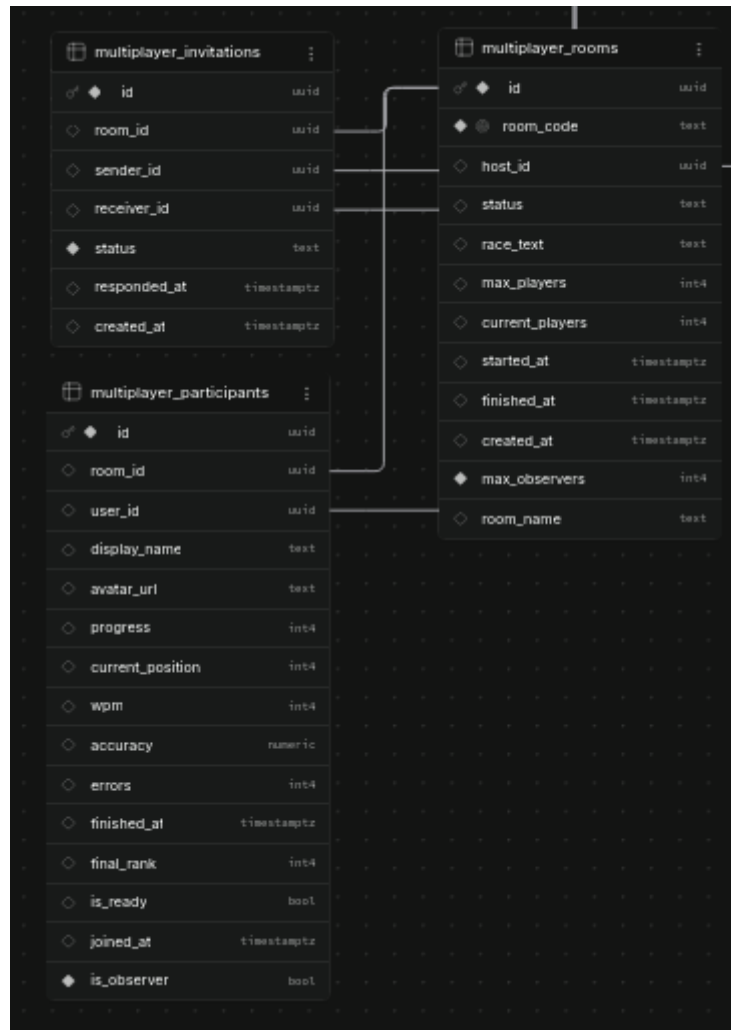
© 2020 KeyDash. All rights reserved.

FAQ

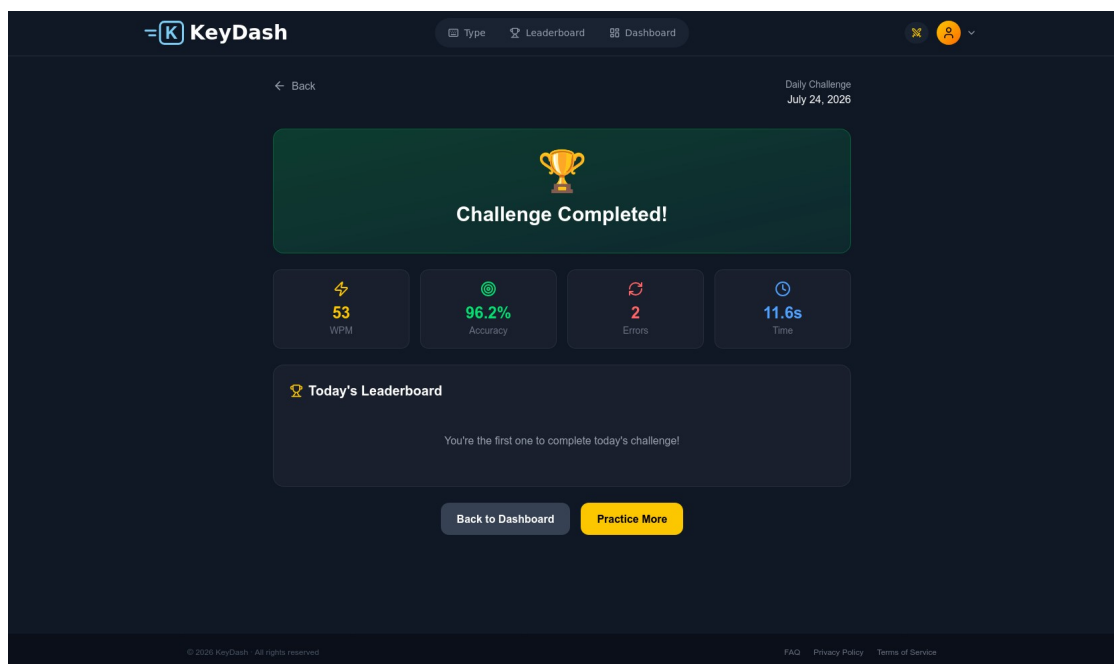
Privacy Policy

Terms of Service

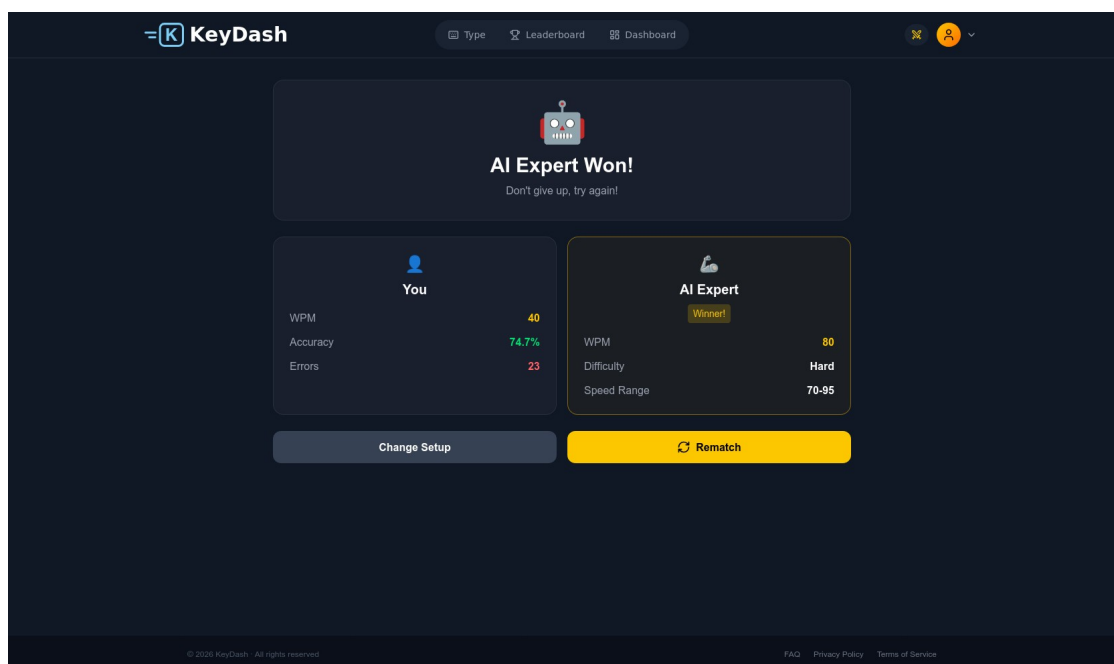
44



Appendix G: Backend Database



Appendix H: Daily Challenge



Appendix I: AI battle